The zlides package

Browser animations between Beamer overlays

Version 0.2, 2026/10/09

Contents

1 Introduction **2**
- 1.1 How the HTML build works . . . 2

2 Requirements and installation **2**
- 2.1 Requirements . . . 2
- 2.2 Installation . . . 3

3 Writing a deck **3**
- 3.1 Loading the package . . . 3
- 3.2 The keys . . . 3
- 3.3 Rules for names . . . 4
- 3.4 A complete example . . . 4
- 3.5 Frame transitions . . . 4

4 The animations in detail **5**
- 4.1 Which animation runs . . . 5
- 4.2 Morph . . . 5
- 4.3 Staged morph . . . 5
- 4.4 Move . . . 6
- 4.5 Camera zoom . . . 6
- 4.6 Fade and draw-on . . . 6
- 4.7 Durations . . . 6

5 Building **6**
- 5.1 The HTML deck . . . 6
- 5.2 Images and figures . . . 7
- 5.3 The PDF . . . 7
- 5.4 Both outputs with latexmk . . . 8

6 Presenting **8**

7 Browser support **8**

8 Limits **8**

9 Development **9**
9.1 Repository layout 9
9.2 Building from the repository 9
9.3 Browser test 9

1 Introduction

zlides adds motion to Beamer slides. You give a name to a TikZ scope in your `.tex` file. The PDF build ignores the names and gives the usual overlays. An overlay is one step of a Beamer frame, and each overlay becomes one page. The `zlides` command builds one HTML file, and the browser animates the named shapes from one page to the next.

You keep one source file with two outputs. The PDF works for print, for handouts, and on Overleaf. The HTML file works for the talk. It contains all pages, all images, and the viewer code, so it works offline in any current browser.

The repository contains an example deck, `examples/showcase.tex`. Its built outputs, `showcase.pdf` and `showcase.html`, are in the same directory. Open the HTML file to see each feature in this manual.

1.1 How the HTML build works

The HTML build has three steps:

1. `latex` compiles the deck to DVI, the TeX output format before PDF. The build selects the `dvisvgm` driver of PGF. With this driver, each named scope becomes an SVG group with a `data-zl-*` attribute.

2. `dvisvgm` converts each DVI page into one SVG file. It draws the text as shapes, so the text looks identical to the PDF.

3. The `zlides` script puts all SVG pages, all images, and the viewer into one HTML file. In the browser, the viewer finds groups with the same name on two consecutive pages and animates between them.

2 Requirements and installation

2.1 Requirements

You need these programs:

- TeX Live or MacTeX. zlides uses `latex`, `dvisvgm`, and `texlua`, which both distributions include.

- For PDF figures only: `pdftocairo` from Poppler, or `pdf2svg`. Neither is part of a TeX distribution.

- A current browser. zlides was tested in Brave, Chrome, Firefox, and WebKit, the engine of Safari.

Without PDF figures, zlides needs nothing outside the TeX installation. It was tested on macOS, and it was not tested on Windows or Linux.

2.2 Installation

Run the install script from the repository:

```
./install.sh
```

The script makes three symbolic links:

- `tex/latex/zlides` into `TEXMFHOME/tex/latex`, for the package.
- `scripts/zlides` into `TEXMFHOME/scripts`, for the build script.
- `scripts/zlides/zlides.lua` as `~/.local/bin/zlides`, for the command.

To put the command in a different directory, set `ZLIDES_BINDIR` before you run the script. If that directory is not on your `PATH`, the script tells you. To remove the three links, run `./install.sh -uninstall`. The script removes each path only if it is a symbolic link.

The repository follows the TeX Directory Structure. To install by hand, copy the `tex` and `scripts` directories into a TeX tree, and make `zlides.lua` executable.

On Overleaf, upload `zlides.sty` into the project. Overleaf then builds the PDF. The HTML build runs only on your computer.

3 Writing a deck

3.1 Loading the package

Load the package after the document class. The package loads TikZ.

```
\documentclass{beamer}
\usepackage{zlides}
```

The deck needs no other change for the HTML build. The `zlides` command selects the SVG driver and passes the `dvisvgm` option to `graphicx` and `xcolor` on the command line.

3.2 The keys

The package defines five TikZ keys. Each key takes a name and goes on a `scope` environment:

`morph=<name>` marks the paths of a shape. On the next page, the paths change into the paths of the scope with the same name.

`move=<name>` marks a group, for example a text node. On the next page, the group moves and scales to the bounding box of the group with the same name.

`zoom=<name>` marks a block for a camera zoom. The block on one page is small, and the block on the next page is large.

`fade=<name>` marks a group that fades in on the first page that has its name.

`drawon=<name>` marks a group whose lines are drawn on, on the first page that has its name.

In the PDF, all five keys do nothing.

3.3 Rules for names

These rules apply to the names:

1. Use each name only once on a page.
2. Give the two shapes the same name on two consecutive pages. The two pages can be two overlays of one frame. They can also be the last overlay of a frame and the first overlay of the next frame.
3. For a morph, put the same number of paths in both scopes. zlides pairs the paths in their order and ignores extra paths.
4. For a camera zoom, put `zoom=<name>` on the small block in the overview and on the large block on the next page. zlides zooms the whole picture that contains the small block.
5. For fade and draw-on, the animation runs on the first page that has the name. If the previous page has the same name, nothing happens.

Keep the same picture size and position on the pages that animate. A `\useasboundingbox` line with the same rectangle in each picture does this. If the picture moves on the page, a block can jump during the animation.

3.4 A complete example

This frame morphs a circle on the first overlay into a rectangle on the second overlay:

```
\begin{frame}{Morph}
  \begin{tikzpicture}
    \useasboundingbox (-1,-1) rectangle (5,2);
    \only<1>{\begin{scope}[morph=box]
      \fill[blue] (0,0) circle (1);\end{scope}}
    \only<2>{\begin{scope}[morph=box]
      \fill[red] (2,0) rectangle (5,2);\end{scope}}
  \end{tikzpicture}
\end{frame}
```

Figure 1 shows the two pages of the PDF. The HTML deck shows the same two states. Between them, the circle moves to the right, changes into the rectangle, and changes color from blue to red.



page 1 page 2

Figure 1: The two PDF pages of the example frame.

3.5 Frame transitions

To set a transition, put one of these commands in the frame:

```
\zltransition{push}
\zltransition{fade}
```

If the deck enters that frame from another frame, the transition runs. A push slides the old page out and the new page in from the side. A fade blends the new page over the old page. Between two overlays of the same frame, no transition runs.

4 The animations in detail

4.1 Which animation runs

On each page change, the viewer runs one main animation. It tries these in order and uses the first one that applies:

1. A camera zoom, if both pages have a `zoom` name with blocks of different sizes.
2. A morph, if both pages have `morph` or `move` names whose shapes differ. The viewer skips a pair whose shapes and colors are identical.
3. A frame transition, if the new page starts a frame with `\zltransition`.

If none applies, the page changes at once. After the main animation, fade and draw-on groups that are new on this page run. If you press a key during an animation, the animation jumps to its end, and the next one starts.

When you go back, the viewer runs the same main animation in reverse. A push moves in the other direction. Fade and draw-on do not run backward.

4.2 Morph

A morph changes the outlines, the fill color, the stroke color, and the line width of each path pair. A path can have several outlines, for example a ring has an outer circle and a hole. The viewer morphs each outline with the outline in the same position of the other path, and it keeps the fill rule. An open outline, for example a plot line, stays open, and its end points map onto each other. The viewer hides only the paths that it morphs, so the rest of the scope, for example a label, stays on screen. The viewer splits each outline into its segments, for example four straight edges and four rounded corners. If the two outlines have different numbers of segments, the viewer splits the longest segments until the numbers are equal. It then places the same number of points on each segment pair. The corners of one shape therefore land on the corners of the other shape. The viewer tries each start segment and both directions, and it uses the match with the smallest total distance.

Between two overlays of one frame, the morph runs alone. The new page fades in over the old page from 35 % to 100 % of the animation.

4.3 Staged morph

Between two frames, the morph runs in three phases:

1. From 0 % to 25 %, the rest of the old picture fades out. The morphing shapes and the moving groups stay.
2. From 20 % to 80 %, the shapes morph and the groups move.

3. From 75 % to 100 %, the new page fades in around the finished shape.

Use the staged morph to zoom into one block of a diagram. Put `morph` on the small block and on a large block that fills the next frame. Put `move` on the label of the small block and on the title of the next frame.

The slide title and the text outside the picture do not fade in phase 1. They cross-fade with the new page in phase 3.

4.4 Move

A move maps the bounding box of the old group onto the bounding box of the new group. The width and the height scale separately. During the move, the old group fades out and the new group fades in, so a text node can change its font size or its color.

4.5 Camera zoom

A camera zoom scales the whole picture that contains the small block. The scale grows until the small block has the size of the large block on the next page. The center of the block moves to the center of the large block at the same time. From 60 % of the animation, the next page fades in. Text outside the picture, for example a caption, does not scale.

4.6 Fade and draw-on

A fade group goes from transparent to opaque. In a draw-on group, each path with a stroke and no fill is drawn from its start to its end. The other parts of the group, for example labels and arrowheads, fade in during the last 40 % of the animation.

4.7 Durations

Table 1 lists the default durations. The `speed` URL parameter multiplies all of them.

Animation	Duration (ms)
Morph inside a frame	700
Staged morph between frames	1300
Camera zoom	1000
Fade transition	450
Push transition	550
Fade and draw-on groups	550

Table 1: Default durations of the animations.

5 Building

5.1 The HTML deck

Run `zlides` with the source file:

```
zlides talk.tex
```

The command writes `talk.html` next to `talk.tex`. It keeps the intermediate files in the directory `talk.zlides/`, so they do not mix with the files of the PDF build. It runs `latex` again while the `.aux` file changes or the log asks for a rerun, with at most three runs in total. If the references still change after three runs, it prints a warning. It then runs `dvisvgm` once.

If your deck needs LuaLaTeX, use the `dvilualatex` engine:

```
zlides --engine=dvilualatex talk.tex
```

The SVG conversion reads DVI, so plain `lualatex` does not work for the HTML build. `zlides` accepts only `latex` and `dvilualatex`.

The HTML build does not run BibTeX or Biber. If the PDF build wrote `talk.bbl` next to `talk.tex`, `zlides` copies it, and the citations show in the HTML deck. Build the PDF first if the deck has citations. Decks that load OpenType fonts through `fontspec` were not tested.

If `latex` fails, `zlides` prints the first error from the log and stops. The full log is `talk.zlides/talk.log`.

5.2 Images and figures

The HTML deck embeds PNG, JPEG, GIF, and SVG images, for example a theme background. zlides finds an image next to the source file or through the TeX search path. `dvisvgm` cannot read PDF files without tools outside TeX, so the HTML build handles figures in these ways:

- PDF figures included with `\includegraphics`: the package lists each PDF and page in `talk.zlides/talk.zlpdf`. `zlides` converts each one to SVG with `pdftocairo` from Poppler, or with `pdf2svg` if `pdftocairo` fails, and runs LaTeX once more to put the SVG files in place. The name of each SVG file contains a hash of the PDF content, so a changed PDF gets a new SVG. A conversion takes about 40 ms for a typical figure. If neither tool is installed, the build stops with an error.
- TikZ pictures that use the `external` library: in the HTML build, zlides turns externalization off, so TikZ draws each picture inline as SVG. The PDF build still uses the cached PDFs.
- Standalone TikZ figures included with `\includestandalone` from the standalone package: in the HTML build, zlides sets the `tex` mode, so the figure source is drawn inline as SVG. The deck must load the packages and macros that the figures need.
- An SVG copy next to the PDF: if `\includegraphics` has no file extension, the HTML build uses the SVG and the PDF build uses the PDF. This path needs no conversion tool.
- EPS figures: the HTML build stops with an error. Convert them to PDF or SVG.

The `trim`, `viewport`, `clip`, and `pagebox` options of `\includegraphics` are not supported for converted PDF figures: the SVG always shows the whole page. For `clip`, zlides prints a warning. If a figure exists as PDF and as EPS, the HTML build uses the PDF. If `dvisvgm` reports an error, `zlides` prints it.

5.3 The PDF

Build the PDF as usual, with `pdflatex`, `lualatex`, or `latexmk`. The keys do nothing in the PDF, and each overlay becomes one page.

5.4 Both outputs with latexmk

latexmk can run `zlides` after each successful PDF build. Add this line to your `latexmkrc`:

```
$success_cmd = 'zlides %S';
```

latexmk runs `$success_cmd` only in watch mode, `latexmk -pvc`. vimtex uses watch mode, so the HTML deck follows each save. Each save then compiles the deck for the PDF and again for the HTML. The file `examples/latexmkrc` shows a complete configuration.

6 Presenting

Open the HTML file in a browser. The deck uses these controls:

- Right arrow, Page Down, Space, or a click: next page.
- Left arrow or Page Up: previous page.
- `f`: full screen on and off.
- `?`: a list of the keys on and off. `Esc` or a click also closes the list.

If the window has a different aspect ratio than the slides, black bars fill the rest of the window. The URL takes these parameters:

`page=N` opens the deck on page N.

`speed=S` multiplies all durations by S. A value below 1 makes the animations faster.

`freeze=T` stops the next main animation at progress T, a number from 0 to 1. The browser test uses it for screenshots.

To present on another computer, copy the HTML file. It needs no other files and no network connection.

7 Browser support

The viewer uses only standard browser features: the SVG geometry functions `getTotalLength`, `getPointAtLength`, `getBBox`, and `getScreenCTM`, the `DOMMatrix` type, and `requestAnimationFrame`. It does not use SMIL animation or the CSS `d` property, which browsers support unevenly. The browser test passed in Chromium, Chrome, Brave, Firefox, and WebKit. Brave and Chrome use the same engine.

8 Limits

zlides has these known limits:

- A morph pairs the outlines of two paths in their order. If one path has more outlines, for example a ring that becomes a disk, the extra outlines do not morph. They show with the new page.
- A morph does not blend dash patterns, gradients, or opacity.

- The HTML deck draws text as shapes, so the viewer cannot select or copy text.
- Text outside the picture does not take part in a staged morph or a camera zoom. It cross-fades with the new page.
- PDF figures need `pdftocairo` or `pdf2svg`, or an SVG copy. EPS figures do not appear in the HTML deck.
- The build was not tested on Windows or with `fontspec` fonts.

9 Development

9.1 Repository layout

The repository contains these files:

`tex/latex/zlides/zlides.sty` the LaTeX package.

`scripts/zlides/zlides.lua` the `zlides` command.

`scripts/zlides/zlides-viewer.js` the viewer that runs in the browser.

`scripts/zlides/zlides-template.html` the page that holds the deck.

`doc/latex/zlides/` this manual.

`examples/` the example deck, its outputs, and its `latexmkrc`.

`tests/check_browsers.py` the browser test.

`tests/check_morph.py`, `tests/check_build.py` the morph test and the build test for PDF figures.

`install.sh` the install script.

9.2 Building from the repository

To build the example deck without installing zlides, run the script with `texlua` from the repository. The script finds `zlides.sty` in the repository by itself.

```
texlua scripts/zlides/zlides.lua examples/showcase.tex
```

To build this manual, run `latexmk` in `doc/latex/zlides`. Its `latexmkrc` adds the package directory to the TeX search path.

9.3 Browser test

The browser test opens a deck in each browser through Playwright. It steps through all pages forward and back, records console errors, and saves one screenshot in the middle of each transition. If a browser reports an error or does not reach the last page and back, the test exits with status 1. The test needs `uv`:

```
uv run --no-project --with playwright \
  playwright install chromium firefox webkit
uv run --no-project --with playwright \
  python tests/check_browsers.py examples/showcase.html shots
```